

UNIT 5 : Introduction to Python

What is Python?

Python is a popular programming language. It was created by Guido van Rossum, and released in 1991.

It is used for:

- web development (server-side),
- software development,
- mathematics,
- system scripting.

What can Python do?

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

Why Python?

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-oriented way or a functional way.

Python Syntax compared to other programming languages

- Python was designed for readability, and has some similarities to the English language with influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.
- Python relies on indentation, using whitespace, to define scope; such as the scope of loops, functions and classes. Other programming languages often use curly-brackets for this purpose.

Python Character Set: Character set is a set of valid characters that a language can recognise. A character represents any digit, alphabet and special symbol. The character used in a Python source program belongs to Unicode standard.

Python supports the following character set:

Alphabets: A to Z (uppercase), a to z (lowercase)

Digits: 0 to 9

Special Characters: ()+-*/&\$#%[]{}!@:;,'"\" etc.

White Space: Blank space, Horizontal tab (→), carriage return, Newline

Other Characters: Python can process any of the ASCII and Unicode characters as data or as literals.

Python Tokens:

A token is the smallest element of a Python program that is meaningful to the interpreter. Each token represents a sequence of characters that can be treated as a single logical unit. The Python recognises these kinds of tokens: identifiers, keywords, literals, operators, punctuators and other separators. A stream of these tokens makes up a translation unit. The programmer can write a Python program by using these tokens. A smallest individual unit in a program is known as token, Lexical unit or Lexical element.

Identifiers: Identifiers are the fundamental building blocks of a program which are used to give the name to data items like: variables, objects, classes, functions, arrays etc.

The rules in Python for identifiers are:

- i. Only alphabets, digits and underscores are permitted.
- ii. Identifiers name cannot start with a digit.
- iii. Keywords cannot be used as identifier.
- iv. Uppercase and lowercase letters are distinct because Python is case sensitive language.
- v. Special characters are not allowed.

Keywords: Keywords are predefined reserved words that have some special purpose or predefined meanings in the programming language. Some of the keywords in Python are: assert, break, class, continue, as, def, if, not, try, in, or, is, return, while, for, with, yield, import, finally, pass, from, False, True, None, and, etc.

Constants or Literals:

Literals are the most obvious kind of constants. They are used to express particular values within the Python program. Literals refer to fixed values that the program may not alter during the execution. Python allows several types of literals which are as follows:

Numeric literals, String literals, Boolean literals, Special literal None, Literal collections.

Operators: Operator is a symbol that operates on operands.

There are two types of operators as follows:

Unary Operators: Those operators that work on single operand to form an expression are known as unary operators. Following are some unary operators:

- + unary plus
- Unary minus
- ~ Bitwise complement
- not logical negation

Binary Operators:

Those operators that work on two operands are known as binary operators.

Punctuators:

Punctuators are also called as separators as it is used to separate line of codes, variables etc. punctuators in Python have syntactic and semantic meaning to the interpreter but do not specify an operation that yields a value. The following characters are used as punctuators in Python: ' " \ [] () { } @ , . : ' =

Expressions:

An expression is a combination of operators, constants and variables as per rule of the language. Any part in a program which can have a value, is an expression. It will have operands and operators together to produce a value. Example: a*5, b+c etc.

Statements:

A statement in Python is a logical instruction, which Python interpreter can read and execute. Example: a=45

Python Comments

- Comments can be used to explain Python code.
- Comments can be used to make the code more readable.
- Comments can be used to prevent execution when testing code.

Creating a Comment

Comments starts with a #, and Python will ignore them:

Example

```
#This is a comment  
print("Good Morning!")
```

Comments can be placed at the end of a line, and Python will ignore the rest of the line:

Example

```
print("Good Morning") #This is a comment
```

A comment does not have to be text that explains the code, it can also be used to prevent Python from executing code:

Example

```
#print("Good Morning")
print("How are you")
```

Multiline Comments: Python does not really have a syntax for multiline comments. To add a multiline comment you could insert a # for each line:

Example

```
#This is a comment
#written in
#more than just one line
print("How are you")
```

Variables

Variables are containers for storing data values.

Creating Variables

Python has no command for declaring a variable.

A variable is created the moment you first assign a value to it.

Example

```
x = 5
y = "Shahid"
print(x)
print(y)
```

Variables do not need to be declared with any particular *type*, and can even change type after they have been set.

Example

```
x = 4    # x is of type int
x = "Basil" # x is now of type str
print(x)
```

Get the Type

You can get the data type of a variable with the `type()` function.

Example

```
x = 5
y = "Shahid"
print(type(x))
print(type(y))
```

Single or Double Quotes?

String variables can be declared either by using single or double quotes:

Example

```
x = "John"
# is the same as
x = 'John'
```

Casting

If you want to specify the data type of a variable, this can be done with casting.

Example

```
x = str(3) # x will be '3'
y = int(3) # y will be 3
z = float(3) # z will be 3.0
```

Variable Names: A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (sal, Sal and SAL are three different variables)
- A variable name cannot be any of the Python keywords.

Input(): This function first takes the input from the user and then evaluates the expression. If the provided input is not correct, then either syntax error or exception is raised by Python.

Syntax:

```
variableName=input(<Message to be displayed>)
```

Example:

```
>>>a=input("Enter any number:")
```

Enter any number: 56

```
>>>print(a)
```

56

print()

This function evaluates the expression before printing it on the monitor. print statement gives output an entire line then goes to next line for subsequent output. Example:

```
print(expression / constant / variable)
```

```
>>>print("Good Morning")
```

```
>>>print(5+6)
```